

TECHCRM · SYSTEM DOCUMENTATION

From signup to closed won

The complete lifecycle of a record in TechCRM — how an organization comes into existence, how a lead is captured and scored, what turns it into an opportunity, and how the deal module reads a meeting and produces a score.

REPOSITORY

D:\CRM · branch main

STACK

React 18 · Spring Boot 4 · PostgreSQL · FastAPI

COMPILED

11 September 2026

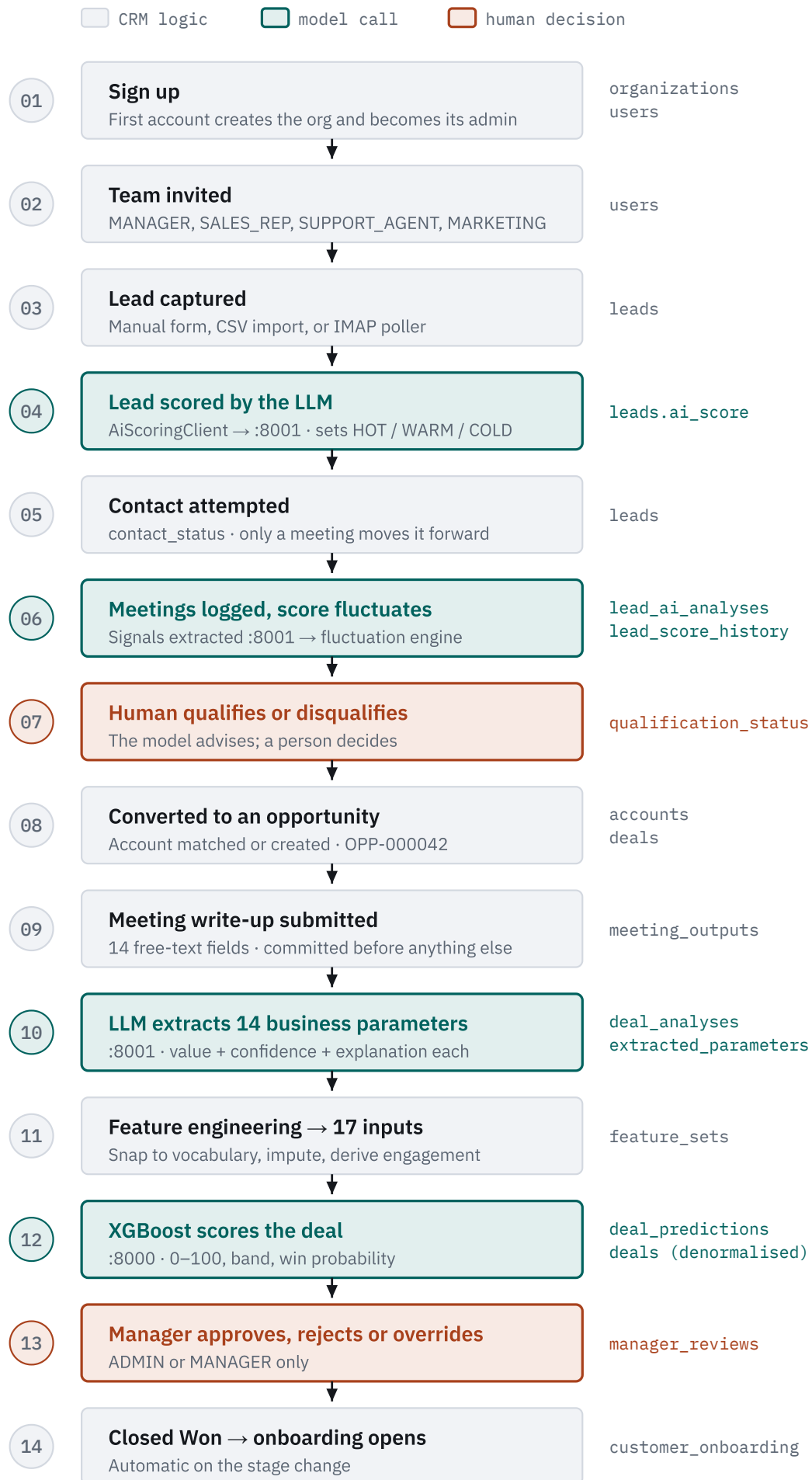
MODELS

Llama 3.1 8B + LoRA · XGBoost

OVERVIEW

The whole path, end to end

Fourteen steps carry a record from an empty database to a signed customer. Everything below is one organization's data: `organization_id` is stamped on every row from step 1 onward, and every query filters on it.



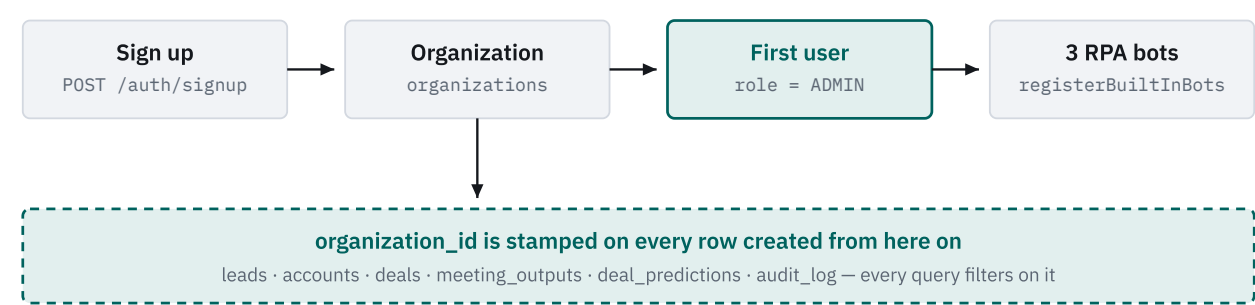
Steps 4, 6, 10 and 12 are the only ones that leave the JVM. Steps 7 and 13 are the only ones a model cannot complete on its own — both are deliberate: the model advises, a person decides.

01

ANONYMOUS VISITOR → ADMINISTRATOR

Organization and user creation

There is no seed data and no tenant provisioning step. The *first* person to sign up brings the organization into existence as a side effect, which is why `AuthService.signup` writes four different kinds of row in one transaction.



Self-service signup, so the admin chooses their own password and `must_change_password` stays false. Users created later by that admin get the flag set instead.

What each step writes

STEP	DETAIL
organizations	Created first — its generated id is what every later row points at.
users	<code>role = ADMIN</code> with <code>PermissionDefaults.forRole(ADMIN)</code> . Duplicate email is rejected before anything is written.
rpa_bots	Three registered automatically: Lead enrichment (on lead created), Follow-up sequencing (hourly), Case routing (on case created).
refresh_tokens	Access JWT lives 30 minutes; the refresh token is an HttpOnly cookie good for 7 days.

Roles, and where they actually matter

ROLE	GATE IT OPENS
ADMIN	Everything, including manager review of deal recommendations.
MANAGER	The only other role that may approve, reject or override a deal prediction (step 13).
SALES_REP	Receives auto-assigned leads; logs meetings and submits deal write-ups.
SUPPORT_AGENT	Cases.
MARKETING	Campaigns.

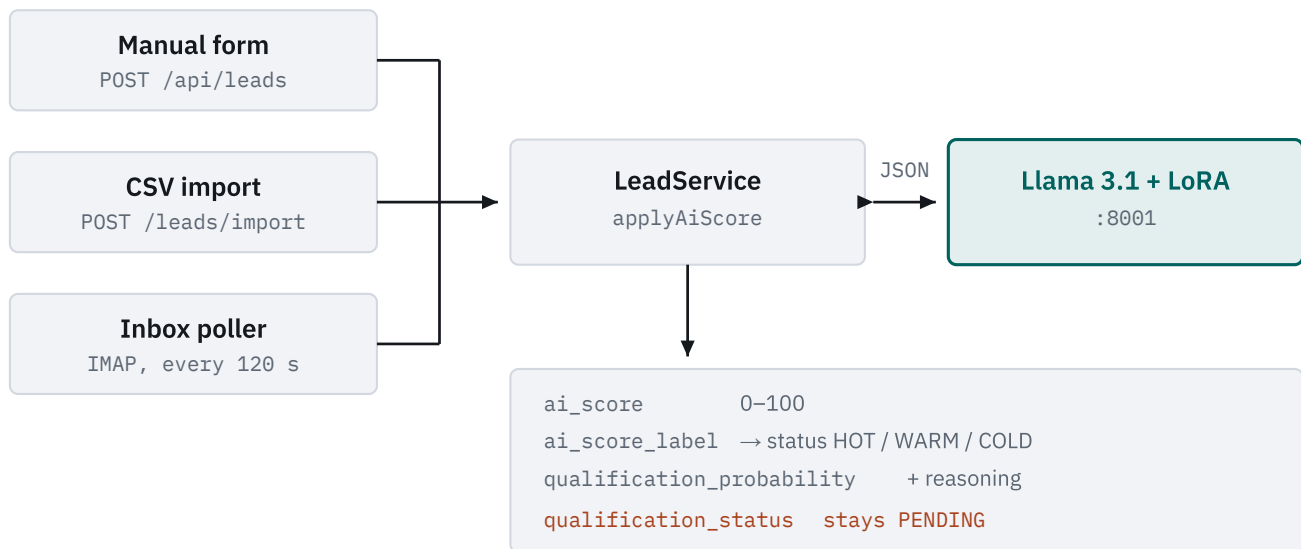
Google and Microsoft sign-in are wired through Spring Security OAuth2. Spring Boot 4 validates every declared registration at startup and **refuses to boot on a blank client id** — so `application-local.yml` must supply them even if nobody ever uses social login.

02

THREE ENTRY POINTS → ONE SCORING PATH

Lead capture and AI scoring

A lead can arrive three ways, but all three land in `LeadService` and all three get scored by the same call. The model's verdict drives the lead's *status*, never its *qualification*.



Re-scoring a lead never overturns a decision a person already made — `applyAiScore` deliberately leaves `qualification_status` untouched in every case.

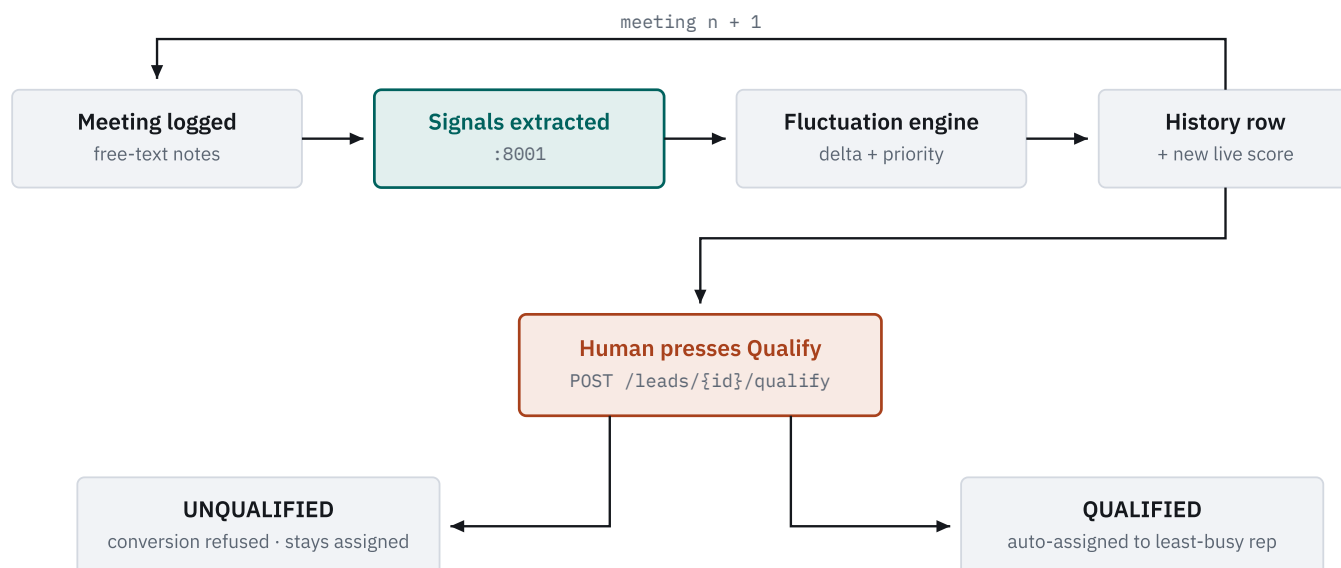
Only three of the label values are honoured. `ai_score_label` is uppercased and matched against `HOT`, `WARM`, `COLD`. Anything else the model returns leaves `status` at whatever it was — typically `NEW` forever. If leads are sticking on `NEW`, that matching is the first place to look.

03

SALES REP, MEETING BY MEETING

The lead loop and qualification

This is a loop, not a step. Each logged meeting is read by the model, scored by the fluctuation engine, and appended to the lead's timeline — the live `ai_score` moves, and the history explains why it moved.



Qualifying is what makes a lead assignable, so it is also the moment the least-busy rep picks it up. Disqualifying deliberately does *not* unassign — that would lose the work already logged against it.

What moves the score

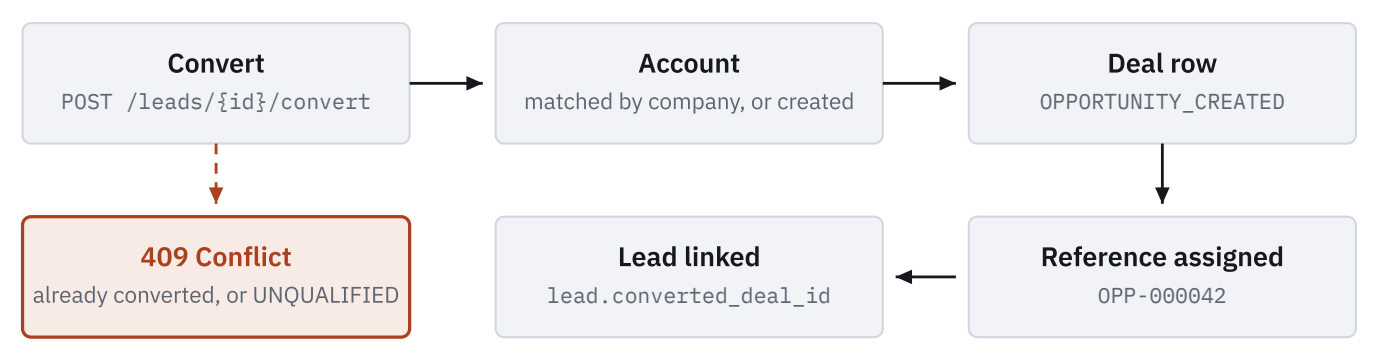
- The reading is recomputed on save, never trusted from the preview call — the notes may have been edited in between, and the preview is a round trip a client could have altered.
- A lead's capture-time score is written lazily as meeting 0, so the timeline starts where the lead actually began instead of springing into existence at its first meeting.
- If the extraction model is unreachable, a keyword heuristic substitutes and the row is tagged `HEURISTIC` instead of `MODEL`. A repaired-JSON reading is tagged `REPAIRED`.

04

LEAD → ACCOUNT + OPPORTUNITY

Conversion

Qualifying and converting are deliberately separate endpoints. A lead marked qualified whose deal creation failed is recoverable; one operation doing both would leave an ambiguous state.



The opportunity reference is assigned *after* the insert, because it is derived from the database-generated id: `OPP-%06d` , zero-padded so references sort.

Two details worth knowing

- Passing a meeting time to the convert call starts the deal at `MEETING_SCHEDULED` instead of `OPPORTUNITY_CREATED` .
- The lead's `estimated_deal_value` and scoring inputs are carried across, so the deal-scoring model starts from what the lead already knew rather than from nothing.

Pipeline stages, in order

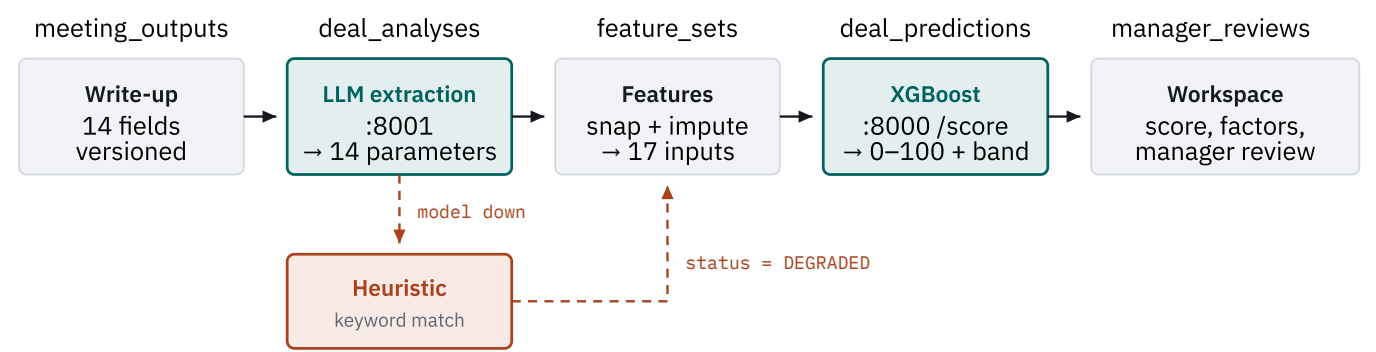
#	STAGE	MEANING
1	OPPORTUNITY_CREATED	Set by conversion.
2	MEETING_SCHEDULED	Set by conversion when a meeting time was supplied.
3	PROSPECTING	Legacy stages, kept rather than renamed — existing deals sit on them, and renaming would rewrite history to match a workflow they were never run through.
4	QUALIFICATION	
5	PROPOSAL	
6	NEGOTIATION	
7	CLOSED_WON	Opens a <code>customer_onboarding</code> record automatically.
8	CLOSED_LOST	Terminal.

05

THE DEAL MODULE

Meeting write-up → deal score

One POST to `/api/deals/{id}/meeting-outputs` runs this entire chain synchronously. Every stage is committed before the next runs, so a failure halfway through leaves a partial chain that can be inspected rather than nothing at all.



Only the write-up is non-negotiable. If the model, the feature layer or the scorer fails, the meeting is still saved — losing the record would be worse than a rough score.

Where the 17 model inputs come from

KIND	COUNT	HANDLING
Categoricals	12	Snapped onto the trained vocabulary — exact, then case-insensitive, then substring. No match falls back to a neutral default and the field is recorded in <code>imputed_fields</code> .
<code>relationship_strength</code>	1	First number wins, so <code>"7"</code> , <code>"7/10"</code> and <code>"7 out of 10"</code> all give 7.0. Clamped 0–10, defaults to 5.
<code>main_objections</code>	1	Split on <code>;</code> or <code>,</code> , each token fuzzy-matched onto the ten known objections, deduplicated, rejoined.
Never extracted	3	<code>total_meetings</code> is the write-up's own version; <code>lead_score</code> comes from the deal; <code>engagement_score</code> is a weighted blend of five already-encoded signals. Asking a language model to invent facts the CRM already knows would be strictly worse than reading them.

Inside the scorer

The Python side never re-derives its own preprocessing. Ordinal maps, objection flags, one-hot columns, the scaler and the final column order all come from the persisted bundle, which is what keeps serving consistent with training. It runs in **strict mode**: an unrecognised label raises rather than silently becoming the median, and the API turns that into a `400`.

OUTPUT	HOW IT IS PRODUCED
deal_score	XGBoost regression, clipped to 0–100. A <code>clipped</code> flag marks predictions that fell outside.
band	≥ 75 HIGH · ≥ 50 MEDIUM · ≥ 25 LOW · otherwise VERY LOW.
win_probability	A logistic squash of the score, tuned so 75 maps to 0.75 — not a trained classifier. No win/loss label was ever in the training data.
confidence	Computed in Java: mean extraction confidence, counting imputed fields as zero so a heavily-defaulted reading reports as less trustworthy.

The deal keeps its old score when the scorer is down. `predictAndPersist` returns early, so the meeting, parameters and features are all saved but no prediction row is written. A stale number beats no number — but the workspace must read `status` to tell the two apart.

06

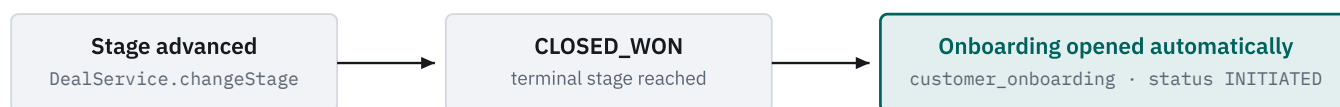
MANAGER → SIGNED CUSTOMER

Review, close, onboard

The model produces a recommended action. A manager is the only thing that can turn it into the deal's *active* action.



The reviewed recommendation is frozen from the prediction row, not taken from the request — so the record shows what the model actually said, not what a client claimed it said.



Asking for a deal's onboarding before it is Closed Won returns an empty body rather than an error — the absence is the answer.

APPENDIX

Processes, and what to start first

PROCESS	PORT	ENTRY POINT	CALLED BY
React SPA	5173	frontend/ · npm run dev	browser
Spring Boot API	8080	backend/ · mvnw spring-boot:run	SPA
PostgreSQL	5432	crm_spring · V1-V21	API
XGBoost scorer	8000	XgBoost/serve_api.py	DealScoringClient
Llama 3.1 + LoRA	8001	Llama3_CRM/scripts/main.py	AiChatClient
Deal-state LoRA	8002	DealIntelligence_CRM/scripts/serve.py	nothing — orphaned

Known gaps

- **The deal-state LoRA is not wired in.** It is trained, served on :8002 and covered by contract tests, but no Java class references `/v1/deal-state` . Deal analysis prompts the general model on :8001 instead.
- **The form scoring path can send empty strings.** `DealScoringClient.score(Deal)` guards only four of the twelve categoricals, then sends `" "` for any null. Strict mode rejects that with a 400 and the deal is silently left unscored.
- **The scoring vocabulary lives in three places** — the frontend constants, `DealParameters` , and the trained bundle behind `GET /schema` . A retrain changes only the third.
- **deal-scoring has no config block.** The client runs on its `@Value` defaults, so the port is invisible to anyone reading `application.yml` .